School of Information and Computer Technology
Sirindhorn International Institute of Technology, Thammasat University
CSS326 Database Programming Laboratory

Laboratory #10: Hardening: Performance & Security

Objectives:

- To explain why queries slow down and how indexes speed them up.
- To create indexes, read a query plan with EXPLAIN and measure the gain with EXPLAIN ANALYZE.
- To weigh the costs of indexing and apply basic query-tuning tips.
- To defend against SQL injection with parameterised queries.
- To apply least privilege: create, check, change and remove database users with GRANT, REVOKE and ALTER USER.
- To protect secrets with a .env file and avoid leaking internal errors.

1. Recap and Goal

Your application works and, since Lab 9, stays correct even when an action spans several statements. Two things still stand between it and the real world: speed and safety. This lab hardens both. The first half is about performance: why queries slow down as tables grow, and how indexes and the query plan make them fast again. The second half is about security: defending against SQL injection, connecting as a least-privilege user instead of root, and keeping secrets out of your code. We use the university schema (students, courses, enrolments) in a new database called css326_lab10.

Before you start, open MySQL Workbench as root and run the three files in the sql folder of Lab10_Exercise_Starter in this order: 01_university_schema.sql creates the tables, 02_university_sample_data.sql adds the sample rows, and 03_test_rows.sql adds 100,000 extra courses so that speed differences become visible.

```
css326_lab10
  students   (id, name, major)
  courses    (id, title, credits, seats)
  enrolments (id, student_id, course_id, grade)
```

2. Why Queries Get Slow

By default, a SELECT ... WHERE has to look at every row to decide which ones match. This is called a full table scan. With fifty rows it is instant, but with five million it is painfully slow, and the cost grows with the number of rows even when only a few of them match. The fix is to let the database jump straight to the rows it needs instead of reading them all, and that is exactly what an index provides.

3. Indexes: What They Are

An index is like the index at the back of a book: a sorted lookup structure that finds entries without reading every page. Instead of scanning the whole table, the database looks the value up in the index and goes straight to the matching rows. MySQL stores indexes as balanced trees (B-trees). Searching

one is like looking up a word in a dictionary: each step skips most of the remaining entries, and every path from the top to the data has the same length, so a lookup takes only a few steps even on a very large table. Your primary key is indexed automatically, which is why finding a row by its id is quick. In this lab we extend that benefit to other columns.

4. Creating an Index

Add an index on the columns you search, join or sort by. The syntax is a single statement. Good candidates are columns that appear in WHERE clauses and columns you ORDER BY. Foreign-key columns are already covered: InnoDB, MySQL's default storage engine, creates an index for every FOREIGN KEY automatically, so enrolments.student_id and enrolments.course_id are indexed already. Use SHOW INDEX to see which indexes a table has.

```
-- speed up searches by course title
CREATE INDEX idx_courses_title ON courses (title);

-- foreign keys (enrolments.course_id) are indexed automatically

SHOW INDEX FROM courses;   -- see what exists
```

5. Reading a Query Plan: EXPLAIN

Before optimising, see how MySQL intends to run a query by putting EXPLAIN in front of it. The output describes the plan, and three columns matter most. type is the access method: ALL means a full table scan (slow), while ref or eq_ref means an index is used. key names the index chosen (NULL means none). rows is how many rows MySQL expects to examine. A type of ALL together with a large rows count is the signal that an index would help.

```
EXPLAIN SELECT * FROM courses WHERE title = 'Operating Systems';

-- key columns in the output:
--   type : ALL = full scan (slow), ref/eq_ref = uses an index
--   key  : which index was used (NULL = none)
--   rows : how many rows MySQL expects to examine
```

6. Before and After an Index

Measure rather than guess: run EXPLAIN, add the index, then run EXPLAIN again and compare. With the 100,000 test rows loaded, the same query that scanned every row (type ALL, key NULL) now uses the index (type ref) and examines a single row. That drop in rows examined is the concrete benefit of the index.

```
EXPLAIN SELECT * FROM courses WHERE title = 'Operating Systems';
--  type: ALL    key: NULL                rows: 100005

CREATE INDEX idx_courses_title ON courses (title);

EXPLAIN SELECT * FROM courses WHERE title = 'Operating Systems';
--  type: ref    key: idx_courses_title    rows: 1
```



7. Measuring the Speed-up

EXPLAIN only shows the plan. EXPLAIN ANALYZE (MySQL 8.0.18 and later) actually runs the query and reports how long each step took. The last number in actual time=a..b is the total time in milliseconds. Speed differences only show on a table with plenty of rows, which is why 03_test_rows.sql adds 100,000 courses. Run each query twice and read the second result, because the first run also loads data from disk. In Workbench, the Duration column of the Output panel gives a quick timing as well.

```
EXPLAIN ANALYZE SELECT * FROM courses WHERE title = 'Operating Systems';

-- before: -> Filter: (courses.title = 'Operating Systems')
--              ... (actual time=0.1..27.4 rows=1)
--            -> Table scan on courses ... (rows=100005)
-- after:  -> Index lookup on courses using idx_courses_title
--              ... (actual time=0.011..0.012 rows=1)
```

8. The Cost of Indexes and Tuning Tips

Indexes are not free. They speed up reads but slow down writes, because every INSERT, UPDATE and DELETE must also update each affected index, and every index takes storage. An unused index is pure overhead, so index the columns you actually query and no more, and confirm the benefit with EXPLAIN. Beyond indexing, a few habits help: select only the columns you need rather than SELECT *, use a composite index when you regularly filter on several columns together, and avoid wrapping an indexed column in a function, because the index stores the column's values, not the function's results.

```
-- fetch only what you need
SELECT id, title FROM courses WHERE title = ?;

-- a composite index for a multi-column filter
CREATE INDEX idx_courses_title_seats ON courses (title, seats);

-- a function on the column stops the index from being used
--   WHERE LEFT(title, 8) = 'Database'   (cannot use the index)
--   WHERE title LIKE 'Database%'        (can)
```

9. Security: the Threats

A database holds your most valuable data, so it is a target. Three risks are especially common. Injection is when a crafted input changes what your SQL actually does. Over-privilege is when the application can do far more than it needs, so a single breach causes far more damage. Exposure is when leaked passwords or detailed error messages give an attacker a map of your system. The rest of this lab addresses all three.



10. SQL Injection and the Parameterised Defence

Building a query by joining user input into the SQL text lets that input rewrite the query. If a search field contains the right characters, the value can break out of its quotes and change the logic, for example turning a lookup into one that returns every row. This is SQL injection. The defence is the parameterised query you have used since Lab 4: send the SQL and the values separately with ? placeholders, so the driver always treats the value as data that can never become code. Always parameterise; never concatenate.

```
// UNSAFE: the value becomes part of the SQL text
const q = "SELECT * FROM courses WHERE title = '" + input + "'";
// input  ' OR '1'='1  turns this into a query that returns every row

// SAFE: value and SQL stay apart
const [rows] = await pool.execute(
  'SELECT * FROM courses WHERE title = ?', [input]);
```

11. Least Privilege: a Dedicated App User

Never let your application connect as root. The root account can drop tables, create users and do anything at all, so if its password leaks, the attacker inherits all of that power. Instead, create a user that can do only what the application needs, on only the application's database: typically SELECT, INSERT, UPDATE and DELETE, and nothing else. This is the principle of least privilege: grant the smallest set of powers that still lets the application work. Then point the application's connection settings at this user.

```
CREATE USER 'university_app'@'localhost'
  IDENTIFIED BY 'a_strong_password';

GRANT SELECT, INSERT, UPDATE, DELETE
  ON css326_lab10.*  TO 'university_app'@'localhost';

-- no DROP, no GRANT, no access to other databases
```

12. Admin and App Users

People and programs need different powers. You, as the administrator, create tables, add indexes and manage users, so an admin account gets ALL PRIVILEGES on the schema. The application only reads and writes rows, so it gets the four data privileges above. root has every privilege on every database (*.*); keep it for setup work only. WITH GRANT OPTION lets a user pass privileges on to other users, so add it only when that is really needed.

```
-- an admin account for people managing the database
CREATE USER 'lab_admin'@'localhost' IDENTIFIED BY 'admin_password';
GRANT ALL PRIVILEGES ON css326_lab10.* TO 'lab_admin'@'localhost';

-- ALL = every privilege on that schema: CREATE, ALTER, DROP, INDEX ...
```



13. Checking and Taking Back Privileges

SHOW GRANTS lists exactly what a user may do. REVOKE undoes GRANT; note that GRANT uses TO while REVOKE uses FROM. A user left with only GRANT USAGE can still log in but cannot touch any data. DROP USER removes the account completely. Changes made with GRANT and REVOKE take effect immediately, so no restart is needed.

```
SHOW GRANTS FOR 'university_app'@'localhost';

-- take one privilege back
REVOKE DELETE ON css326_lab10.* FROM 'university_app'@'localhost';

-- take everything back (the user can still log in, but do nothing)
REVOKE ALL PRIVILEGES ON css326_lab10.* FROM 'university_app'@'localhost';

-- remove the user completely
DROP USER 'university_app'@'localhost';
```

14. Changing a User

ALTER USER changes an existing account without deleting it. Use it to set a new password for another user, to change your own password, or to lock an account so that it cannot log in while keeping its privileges for later. RENAME USER changes the account name. After changing the application's password, update it in the application's settings too, or the application will no longer be able to connect.

```
ALTER USER 'university_app'@'localhost' IDENTIFIED BY 'new_strong_password';
ALTER USER USER() IDENTIFIED BY 'my_new_password';   -- your own password

ALTER USER 'university_app'@'localhost' ACCOUNT LOCK;     -- cannot log in
ALTER USER 'university_app'@'localhost' ACCOUNT UNLOCK;

RENAME USER 'university_app'@'localhost' TO 'uni_app'@'localhost';
```

15. Switching Between Users

In MySQL Workbench, each user is a separate connection. Click the + next to MySQL Connections, give the connection a name, set Username to university_app, and click Test Connection. Open that connection to work as that user; your root connection stays as it is. In the application, the user is whoever db.js logs in as (after Exercise 3, the values in .env); save the file and npm run dev restarts the server. SELECT CURRENT_USER() always tells you who you are connected as.

```
SELECT CURRENT_USER();      -- who am I connected as?
DROP TABLE drop_test;       -- as university_app:
-- ERROR 1142 (42000): DROP command denied to user 'university_app'@'localhost'
```

16. Protecting Secrets and Errors

Since Lab 6, the database password has been typed straight into the code (db.js). If that file is pushed to GitHub, anyone who opens the repository can read the password, and every machine (your laptop, a teammate's, the real server) needs different settings, which means editing code each time. A .env file solves both problems: it keeps the settings outside your code, each machine has its own copy, and it never leaves the machine. Install the dotenv package once with npm install dotenv, read the values with process.env, and add .env to .gitignore so that Git never sees it.

```
# .env   (project root, never committed)
DB_HOST=localhost
DB_USER=university_app
DB_PASS=a_strong_password
DB_NAME=css326_lab10

# .gitignore
.env
node_modules/
```

Then read the values in db.js:

```
// db.js   (require dotenv on the very first line)
require('dotenv').config();
const mysql = require('mysql2/promise');

const pool = mysql.createPool({
  host:     process.env.DB_HOST,
  user:     process.env.DB_USER,
  password: process.env.DB_PASS,
  database: process.env.DB_NAME,
});
module.exports = pool;
```

Finally, do not send raw database errors or stack traces back to the client, because they reveal your table names and structure. Log the details on the server with console.error and return a short, generic message instead. Keep validating input as in Lab 6 as well: security works in layers.

17. Good Habits

Index the columns you actually query (those in WHERE, JOIN and ORDER BY) and confirm the benefit with EXPLAIN and EXPLAIN ANALYZE rather than guessing; do not over-index, since every write pays the cost. Parameterise every value, which gives you correctness and security at the same time. Connect as a least-privilege user rather than root, check privileges with SHOW GRANTS, keep secrets in .env, hide internal errors behind generic messages, and keep validating input. Treat performance and security as ongoing practices that you measure and layer, not as one-off fixes.

18. Lab Exercises (complete during class)

Getting started

Download and unzip **Lab10_Exercise_Starter.zip**. It contains:

```
Lab10_Exercise_Starter/
  db.js                 connection pool (root for now; Exercise 3 changes it)
  server.js             Express app + front-end on http://localhost:3000
  public/index.html     the Lab 10 web app
  routes/               courses.js, students.js
  controllers/          course-, student- and toolsController.js
  sql/                  01_university_schema.sql      creates css326_lab10
                        02_university_sample_data.sql sample rows
                        03_test_rows.sql              100,000 extra courses
                        exercise1_index_explain.sql, exercise2_app_user.sql,
                        exercise4_tuning.sql
  requests.http         ready-made test requests
```

- In Workbench, as root, run `sql/01_university_schema.sql`, `02_university_sample_data.sql` and `03_test_rows.sql` in this order (Section 1).
- Open `db.js` and put your own root password for now. In Exercise 3 you replace it with `university_app` and a `.env` file.

Then open a terminal **in the project folder** (in VS Code: Terminal → New Terminal) and run the two npm commands below.

```
npm install      # once: downloads express and mysql2 into node_modules/
npm run dev      # starts the server
                 #   > node --watch server.js
                 #   Lab 10 server on http://localhost:3000
                 # Ctrl + C stops the server
```

`npm run dev` runs `node --watch server.js`, which **restarts the server every time you save a .js file** (or `.env`, once `db.js` reads it), so you never have to stop and start it by hand. Leave the terminal open while you work.

- Open **http://localhost:3000**. The page has five panels: **Connection** (which MySQL user the app logs in as, and its privileges from SHOW GRANTS), **Query plan** (EXPLAIN for the course search and the indexes on courses), **Course search** (with the server time of every request and one-click attack buttons), **A student's courses** (the injection by id) and a **Responses** log.
- The log turns **red** when something is unsafe: an injection that worked, or a raw database error sent to the browser. After Exercise 3 there should be no red lines, and the Connection panel should show `university_app` with only the two GRANT lines.
- You can also test with `requests.http` (VS Code REST Client) or Postman.

If the server doesn't start, read the first error line in the terminal:

Error message	What it means / what to do
`Cannot find module 'express'`	You skipped `npm install`, or ran it in a different folder.
`Cannot find module 'dotenv'`	Exercise 3: run `npm install dotenv` in the project folder.

Error message	What it means / what to do
`Access denied for user 'root'`	Wrong password in `db.js` (later: `DB_PASS` in `.env`).
`Access denied for user ''@'localhost'`	`.env` was not found: name it exactly `.env` (not `.env.txt`), in the project folder.
`Unknown database 'css326_lab10'`	Run `sql/01_university_schema.sql` in Workbench first.
`EADDRINUSE … :3000`	The server is already running in another terminal; stop it with Ctrl + C.

Exercise 1: Index and EXPLAIN (exercise1_index_explain.sql). Pick a query you run often with a WHERE on a non-key column, for example courses.title. Run EXPLAIN and EXPLAIN ANALYZE and note the type, rows and actual time. Create an index on that column, run both again, and record how the three values changed. This is your evidence that the index helped.

In the web app you can watch the same change: the **Query plan** panel turns from type **ALL** (red) to **ref** (green) and lists the new index, and the time shown after each search drops sharply. Run each search twice and compare the second times.

Exercise 2: A least-privilege app user (exercise2_app_user.sql). Create the user university_app and GRANT it only SELECT, INSERT, UPDATE and DELETE on css326_lab10. Check the result with SHOW GRANTS, REVOKE DELETE, check again, then GRANT it back. Open a new Workbench connection as university_app, confirm who you are with SELECT CURRENT_USER(), and check that the user cannot DROP the scratch table drop_test.

Exercise 3: Harden the app (the Lab 10 web app). The starter application works but is unsafe. First press the attack buttons in the web app (' OR '1'='1 and a single quote in Course search, 1 OR 1=1 in A student's courses) and watch the red lines in the Responses log. Then fix the two queries that build SQL by hand so that they use ? placeholders, switch the connection to university_app, move the database settings into a .env file read with process.env (and add .env to .gitignore), and make sure a failed request returns a clean message rather than the raw database error. Try the same input again after your fix.

Exercise 4: Tuning, if time allows (exercise4_tuning.sql). Add a composite index for a query that filters on two columns at once and use EXPLAIN ANALYZE to confirm that it is used. Compare selecting only the columns you need against SELECT *, and compare a function on an indexed column with a LIKE prefix match.

19. Where We're Going Next

Your application is now correct, fast and reasonably hardened, which is close to how a real production system is built. Next we explore features that live inside MySQL itself. In Lab 11 you will write stored procedures, functions and triggers that run in the database, and meet MySQL's NoSQL side: the JSON data type and the X DevAPI document store.